



SmartLogix

Plataforma inteligente para la gestión logística de eCommerce.

Examen Final Transversal · Desarrollo Fullstack III (DSY1106)

Gabriel Jeria · Vicente Palacios · Isak Chacana

Sección 003D · Docente: Claudio Rojas Rodríguez

Introduccion.

Principales Problemas.

1

Sincronización deficiente

Inventarios desactualizados entre sistemas manuales y monolíticos.

2

Errores en pedidos

Asignación incorrecta de órdenes por falta de integración en tiempo real.

3

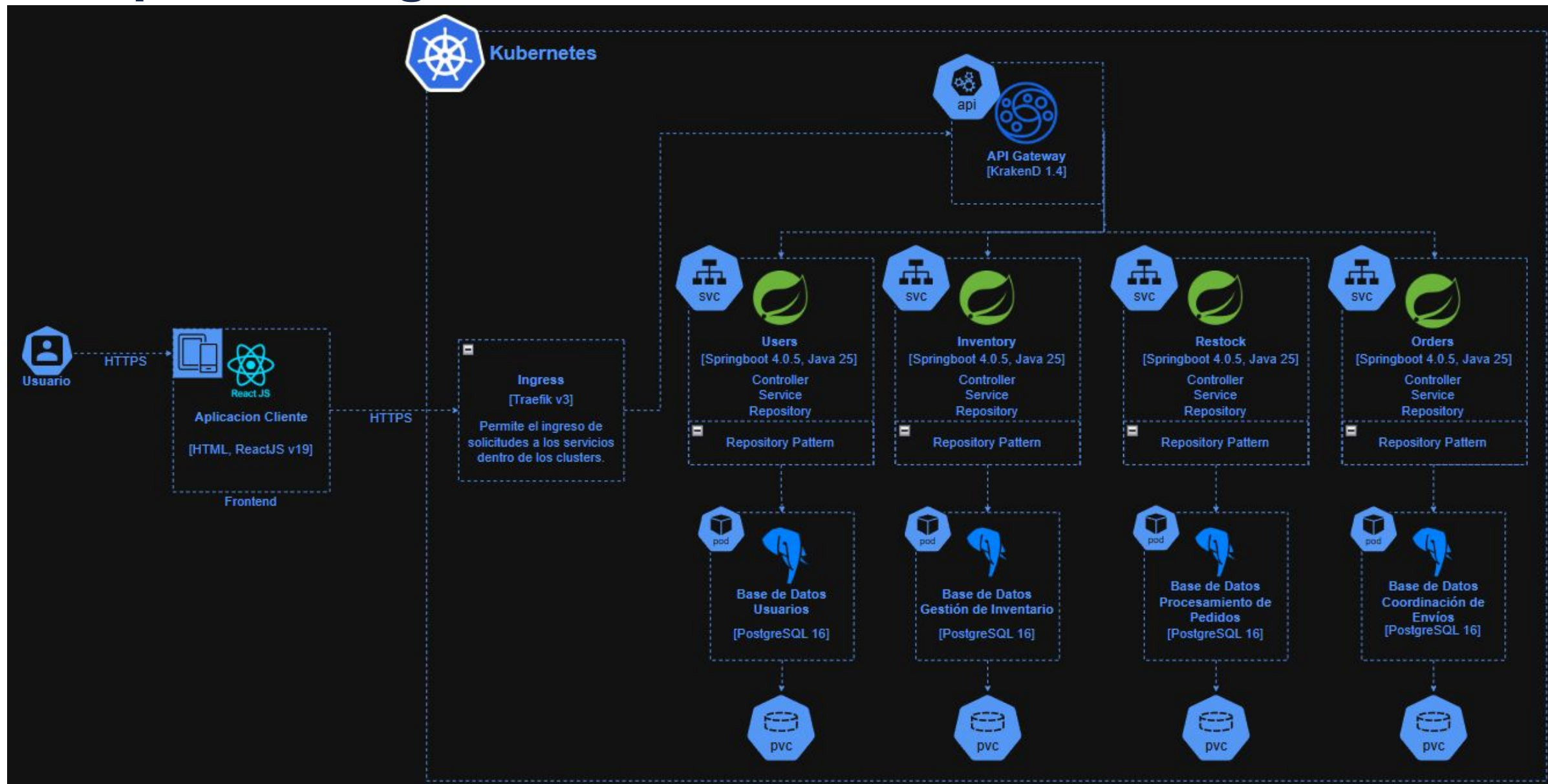
Escalabilidad limitada

Los sistemas monolíticos no soportan picos de demanda del eCommerce.

Solucion

Una arquitectura de microservicios en Kubernetes que permite a PYMEs de eCommerce gestionar inventario, pedidos y reposición de stock de forma escalable, segura y automatizada.

Arquitectura general



Patrones de diseño aplicados

MSC

Modelo-Servicio-Controlador

Arquetipo base en los 4 microservicios: Controller expone REST, Service concentra lógica, Repository accede a datos.

Repo

Repository Pattern (JPA)

Cada microservicio abstrae su acceso a PostgreSQL vía Spring Data JPA, aislado del resto del sistema.

DTO

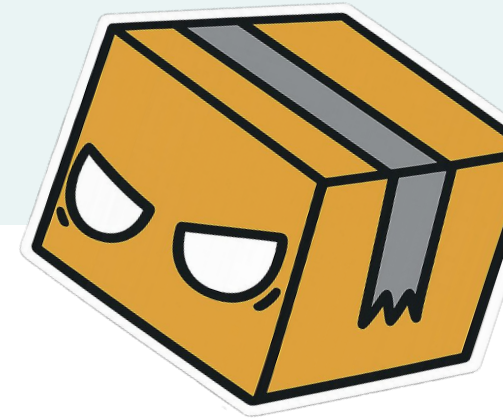
Data Transfer Object

Transporta datos entre frontend, BFF y microservicios sin exponer entidades internas.

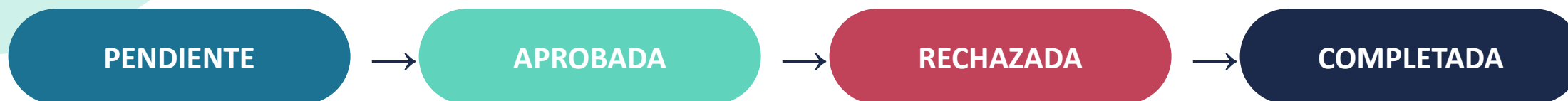
KD

KrakendID

API Gateway declarativo que enruta cada endpoint público directo al microservicio correspondiente, validando JWT y aplicando circuit breaker por configuración.



Caso destacado: ms-restock



Integración con ms-inventory

Vía RestTemplate: valida que el ítem exista y, al aprobarse, actualiza el stock automáticamente sumando la cantidad solicitada.

Persistencia propia

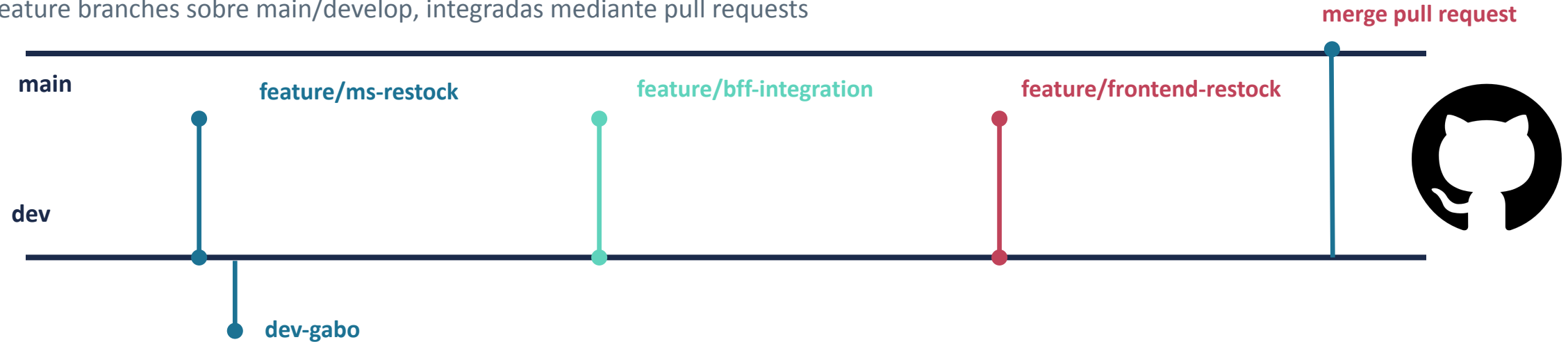
Base PostgreSQL dedicada (restock-db), gestionada con Flyway, respetando la autonomía de datos por microservicio.

Exposición vía API GATEWAY

KrakenD expone el servicio en /api/bff/restock, enrutando directamente hacia ms-restock, sin intermediario de código.

Estrategia de branching

Feature branches sobre main/develop, integradas mediante pull requests



Lo que funcionó

Trabajo paralelo en ms-restock, BFF y frontend sin bloquear a los demás integrantes del equipo.

Desafío enfrentado

Conflictos de merge al modificar simultáneamente archivos de configuración compartidos (application.properties, pom.xml).

Mejora a futuro

Convenciones más estrictas de nombres de rama y code review obligatorio antes de cada integración.

Pruebas unitarias

JUnit 5 + Mockito sobre ms-restock

60%+

cobertura en la capa de
servicio de ms-restock
(JaCoCo)

JWT

validación de tokens y
manejo global de
excepciones probados

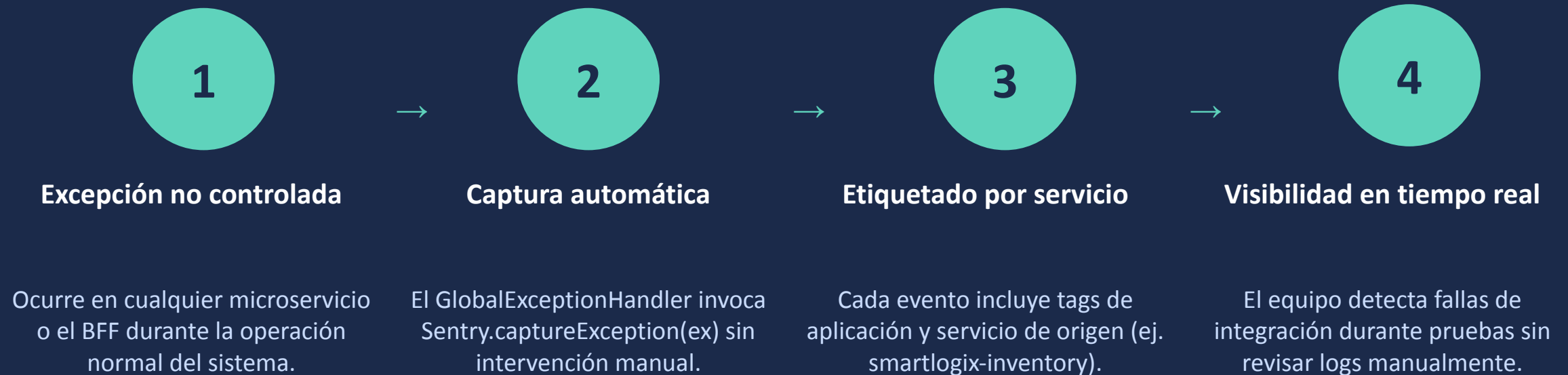
Enfoque técnico

@ExtendWith(MockitoExtension.class), @Mock e @InjectMocks para simular la capa de repositorio y RestTemplate, verificando el ciclo de vida de las solicitudes de restock (pendiente, aprobada, rechazada, completada), incluyendo escenarios de error.



Monitoreo y observabilidad

Sentry (autoalojado sobre GlitchTip) en todos los microservicios.



GlitchTip

Ética, privacidad y sostenibilidad.

Escalabilidad responsable

Kubernetes escala solo los servicios que lo requieren (ej. Inventory en picos), evitando el sobreaprovisionamiento de recursos de un monolito.

Privacidad de datos

Credenciales y tokens se gestionan vía JWT (emitido por ms-user); los microservicios no se exponen directamente, solo a través del API Gateway (KrakenD)

Mejora identificada

Incorporar cifrado explícito (HTTPS/TLS) en la comunicación interna entre microservicios, hoy protegida solo por el aislamiento de red de Kubernetes.



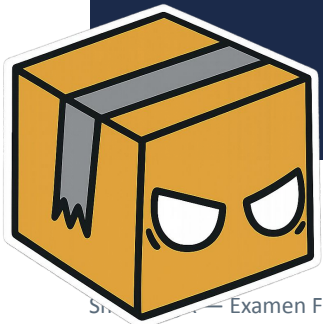
Retrospectiva del equipo

Qué funcionó bien

- Separación en capas del frontend (Api / ViewModels / Pages)
- Autonomía de datos por microservicio con Flyway
- RestTemplate centralizando la integración de ms-restock con ms-inventory
- Detección temprana de errores gracias a Sentry

Qué mejoraríamos

- Librería compartida para reducir boilerplate entre microservicios
- Mapeo automático de DTOs (MapStruct) entre capas
- Convenciones de branching más estrictas y code review obligatorio
- Pruebas de integración con Testcontainers, no solo mocks





¡Gracias por su atención!